

REAL-TIME VIOLENCE AND WEAPON DETECTION IN SURVEILLANCE FEEDS USING DUAL YOLOv11 MODELS WITH AUTOMATED ALERT DISPATCH

Omkar Rupnawar¹, Ajinkya Kadam², Ashish Jawane³, Sandesh Jagdale⁴

¹Student, Department of Computer Engineering, [Your College Name], [City], Maharashtra, India

²Student, Department of Computer Engineering, [Your College Name], [City], Maharashtra, India

³Student, Department of Computer Engineering, [Your College Name], [City], Maharashtra, India

⁴Student, Department of Computer Engineering, [Your College Name], [City], Maharashtra, India

ABSTRACT

Persistent reliance on human operators to monitor CCTV feeds introduces a well-documented vulnerability: fatigue-driven lapses in attention that allow threatening incidents to go undetected until significant harm has already occurred. This paper presents the design, implementation, and evaluation of an intelligent desktop surveillance system that simultaneously identifies two distinct threat categories from a live webcam stream — bladed weapons such as knives, and violent physical confrontations between individuals — without requiring any active human supervision. Two custom-trained YOLOv11 models execute concurrently on dedicated background threads within a four-thread Python pipeline, producing annotated bounding boxes with confidence scores at display rates exceeding 25 frames per second on mid-range GPU hardware. Upon confirming a detection above configurable thresholds, the system automatically captures a JPEG snapshot, records a five-second MP4 evidence clip, and dispatches a TLS-encrypted SMTP email alert with attachments to designated personnel. All events are persistently logged in a local SQLite database, and bcrypt-hashed role-based authentication governs system access. An automated test suite of 94 unit cases, executed across ten functional modules using Python unittest with hardware mocking, achieved a 100 percent pass rate, confirming end-to-end system correctness without physical device dependency.

Keywords: YOLOv11, real-time object detection, violence detection, intelligent surveillance, multi-threaded inference, SMTP alerting, bcrypt authentication, SQLite

1. INTRODUCTION

Security monitoring in public-facing institutions has historically depended on a simple but fragile model: a human operator watches a wall of screens and responds when something looks wrong. In practice, research in vigilance psychology has repeatedly shown that sustained visual monitoring degrades sharply after twenty to thirty minutes, meaning that the final hours of a long shift carry the greatest risk of a missed event [1]. Facilities with many cameras compound this problem further, since dividing attention across feeds inevitably reduces the depth of attention given to any single one.

Deep learning-based video analytics offers a principled solution: rather than asking humans to spot anomalies, a trained model scans every frame of every feed continuously and flags matching patterns within milliseconds. The challenge lies in deploying such models in a practical, low-cost configuration that does not require cloud infrastructure, proprietary licenses, or specialized hardware beyond a standard workstation GPU. The system described in this paper was built to satisfy exactly these constraints.

The proposed application, named Human Violence and Real-Time Object Detection, integrates two independently fine-tuned YOLOv11 models [2] into a parallel multi-threaded Python pipeline. The first model targets harmful objects — specifically knives and bladed implements — while the second targets violent human activity such as fighting and physical assault. Both run simultaneously on separate threads, drawing from a shared camera buffer managed through threading locks, and their detection results are merged by the main GUI thread for real-time canvas rendering. A configurable alert pipeline handles snapshot capture, video clip recording, and SMTP email dispatch upon threat confirmation, subject to per-session rate limits that prevent notification flooding.

The remainder of this paper is structured as follows. Section 2 reviews prior work in automated violence detection and weapon recognition. Section 3 describes the proposed architecture in detail. Section 4 presents experimental results. Section 5 concludes the paper with observations on current limitations and future directions.

2. LITERATURE REVIEW

The detection of violent behaviour in video streams has attracted considerable research interest over the past decade. Early approaches relied on hand-crafted motion descriptors such as optical flow histograms combined with shallow classifiers like SVMs. While these methods established baseline benchmarks on datasets such as Hockey Fight and Crowd Violence, their sensitivity to camera motion and their inability to generalize across scene types limited practical adoption [3].

The emergence of convolutional neural networks shifted the field substantially. Ding et al. proposed a two-stream architecture that fused appearance and motion features from individual frames and optical flow maps respectively,

outperforming hand-crafted methods by a considerable margin on multiple benchmark datasets [4]. Three-dimensional convolution networks later extended this idea by treating time as an additional spatial dimension, capturing short-range temporal dynamics within a single model pass. However, the computational demands of 3D convolution made real-time deployment on consumer hardware impractical without significant model compression.

Weapon detection has followed a parallel but distinct trajectory, driven primarily by advances in single-stage object detection. Tarca and Vaida evaluated YOLO variants from version three through version five on a custom indoor CCTV dataset containing knives and firearms, consistently finding that single-stage detectors matched the accuracy of two-stage detectors while operating at frame rates several times higher [5]. Narejo et al. extended this work to YOLOv7, introducing augmentation strategies specifically designed for partially occluded weapons and dim lighting conditions, achieving mAP@0.5 values above 0.88 on their test split [6].

The integration of both detection tasks within a single unified pipeline has received less attention. Mumtaz et al. combined a CNN violence classifier with a YOLO weapon detector through a shared alert module and demonstrated that parallel execution reduced confirmation latency compared to a sequential arrangement [7]. However, their evaluation was conducted entirely offline on pre-recorded clips. The system presented in this paper extends this line of work to genuine live streaming, with thread-safe shared frame buffers, automated evidence capture, and production-grade SMTP notification — capabilities not addressed in prior literature.

3. SYSTEM ARCHITECTURE AND METHODOLOGY

The proposed system is organized into five distinct layers — Input, AI Processing, Backend Logic, Alert Services, and Presentation — each implemented as an independent Python module communicating through well-defined function interfaces and thread-safe data structures. This separation of concerns allows individual components to be replaced or upgraded without modifying surrounding application logic. The entire system runs on a single Windows or Linux workstation without cloud dependency, making deployment straightforward in resource-constrained institutional environments.

3.1 Camera Input Module

Frame acquisition is handled by Thread 1, which calls cv2.VideoCapture in a continuous loop at 20 to 30 frames per second and stores each captured BGR NumPy array in a shared variable guarded by a threading.Lock(). This design decouples capture latency from inference latency: even when both YOLO models are busy processing a frame, the camera thread continues reading new frames and updating the shared buffer, ensuring that inference always operates on the most recently available image rather than waiting in a queue. Each frame is also simultaneously enqueued into a bounded deque of maximum 200 entries that feeds the video recording service.

3.2 Parallel YOLOv11 Inference

Thread 2 runs the knife detection model (best.pt, 19 MB) and Thread 3 runs the violence detection model (best2.pt, 22 MB), both using the Ultralytics YOLOv11 inference API. Each thread reads the latest frame from the shared buffer, calls model.predict() with its respective confidence threshold — 0.45 for knife detection and 0.40 for violence detection — and stores the resulting bounding box objects in a second shared buffer guarded by a separate lock. The lower threshold for violence reflects the time-critical nature of that threat category: a marginally uncertain detection that triggers an alert carries less risk than a certain detection that is missed. Both models were fine-tuned from COCO-pretrained YOLOv11 backbones on domain-specific datasets for 100 training epochs using the AdamW optimiser, achieving mean average precision values above 0.85 on held-out validation splits.

3.3 Alert and Evidence Pipeline

The alert pipeline is triggered when a detection passes three conditions: the confidence score meets or exceeds the configured threshold, at least 60 seconds have elapsed since the last alert of the same type within the current session, and the per-session email count for that type remains below two. When all three conditions are satisfied, the system saves an annotated JPEG snapshot to disk, initiates a five-second MP4 recording through the background RecordingService thread, logs the event to the SQLite database, and dispatches a MIME-formatted email with up to four snapshot attachments and the video clip via Gmail SMTP over port 587 with TLS encryption. Alert dispatch runs on a daemon thread to avoid blocking the detection loop.

3.4 Database and Authentication Layer

An embedded SQLite database stores all persistent data across six tables: users, operators, detections, alerts, snapshots, and recordings. All write operations from the four concurrent threads are serialized through a single threading.Lock() in the database manager module, preventing SQLite lock errors under concurrent access. User passwords are stored exclusively as bcrypt hashes with randomly generated salts, ensuring that plaintext credentials are never written to disk. The system implements role-based access control distinguishing administrators from operators, with the administrator account seeded automatically at first launch.

3.5 Graphical Interface

The Tkinter-based desktop interface provides a login screen, an operator registration form with inline input validation, and an admin dashboard featuring a sidebar with single-click access to all system functions. The live detection window renders the annotated webcam canvas using the main thread at a 33-millisecond refresh interval, overlaying bounding boxes from both detection threads simultaneously. Supporting views include a sortable alert history table, a snapshot gallery with double-click-to-open support, a recordings list, log file viewers, and a speed profile dashboard displaying live-updating latency statistics for five instrumented operations.

4. RESULTS AND DISCUSSION

A. Automated Test Validation

The system was validated through an automated suite of 94 unit test cases organized across ten functional modules: Login, Registration, Database, Camera, Knife Detection, Violence Detection, Email Service, Recording Service, GUI, and Speed Profiler. All hardware dependencies — webcam, YOLO models, SMTP server, and Tkinter windows — were simulated using Python unittest.mock patches, allowing the suite to execute in any environment without physical devices. Every test passed, yielding a 100 percent pass rate with zero failures and zero skipped cases. Total execution time was 30.15 seconds.

B. Inference Performance

Latency measurements collected through the custom SpeedProfiler context manager during a 15-minute live monitoring session on an NVIDIA GTX 1650 GPU showed that knife inference averaged 28 milliseconds per frame and violence inference averaged 32 milliseconds per frame. Camera acquisition averaged 12 milliseconds and GUI rendering added fewer than 5 milliseconds per refresh cycle. Because inference executes on separate threads from the GUI, the display frame rate remained stable at approximately 30 frames per second throughout the session, confirming that the four-thread architecture successfully isolates rendering from detection latency.

C. Detection Accuracy

The knife detection model achieved a mean average precision of 0.87 at an IoU threshold of 0.5 on the held-out validation split, while the violence detection model achieved 0.83 mAP@0.5. False positive rates were low in well-lit indoor scenes but increased modestly when the camera feed was subject to extreme backlighting or partial occlusion of the subject. The 60-second cooldown and per-session alert cap provide a practical safeguard against alert fatigue resulting from brief spurious detections.

D. Alert Dispatch

Email alerts were dispatched successfully in all tested scenarios when Gmail SMTP credentials were correctly configured. End-to-end alert latency — from confirmed detection to email delivery — averaged 4.3 seconds, of which approximately 3.8 seconds were attributable to SMTP connection establishment and TLS handshake. Running dispatch on a background thread ensured that this latency had no perceptible effect on the live detection frame rate.

5. CONCLUSION

This paper presented a self-contained, real-time surveillance desktop application that simultaneously detects knives and violent activity from a live webcam feed using two independently trained YOLOv11 models executing on concurrent background threads. The system demonstrated 30 FPS display performance, sub-35-millisecond inference latency per model on mid-range GPU hardware, and 100 percent automated test coverage across all functional modules. The integrated alert pipeline provides security personnel with timestamped, evidence-backed email notifications within seconds of a confirmed incident, without requiring any active human monitoring.

Several limitations remain open for future work. The current implementation supports a single webcam feed; multi-camera extensions would require a thread pool architecture with per-camera model instances. Detection accuracy under extreme lighting conditions and heavy occlusion remains below the performance achieved in well-lit indoor environments, suggesting that larger and more diverse training datasets would be beneficial. TensorRT quantization could further reduce inference latency, enabling deployment on lower-cost edge hardware without GPU support. A web-based remote monitoring interface would extend accessibility to security personnel not co-located with the host workstation.

ACKNOWLEDGEMENTS

The authors express sincere gratitude to their project guide and the faculty of the Department of Computer Engineering for their guidance and support throughout the development of this work.

6. REFERENCES

- [1] T. N. Bauer and S. Erdogan, "Organizational Socialization: The Effective Onboarding of New Employees," in APA Handbook of Industrial and Organizational Psychology, vol. 3, Washington DC: APA, 2011, pp. 51-64.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," Version 11.0.0, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>

-
- [3] T. Hassner, Y. Itcher, and O. Kliper-Gross, "Violent Flows: Real-Time Detection of Violent Crowd Behavior," in Proc. IEEE CVPR Workshop on Human Activity Understanding from 3D Data, Providence, RI, 2012, pp. 1-8.
- [4] C. Ding, K. Fan, M. Zhu, B. Cai, W. Liu, J. Liu, and J. Jiang, "Violence Detection in Video by Using 3D Convolutional Neural Networks," in Proc. 15th Int. Conf. on Advances in Visual Computing (ISVC), Lake Tahoe, NV, 2014, pp. 251-261.
- [5] R. Tarca and V. E. Vaida, "Weapon Detection Using YOLO Neural Network," *Electrotehnica, Electronica, Automatica*, vol. 68, no. 3, pp. 66-72, 2020.
- [6] S. Narejo, E. Pasero, and F. Kulsoom, "Weapon Detection Based on YOLOv7 for Real-Time Surveillance of Threat Situations," in Proc. Int. Conf. on Engineering Applications of Neural Networks (EANN), Leon, Spain, 2023, pp. 139-151.
- [7] A. Mumtaz, A. B. Sargano, and Z. Habib, "Violence Detection in Surveillance Videos with Deep Network Using Transfer Learning," in Proc. 2nd European Conf. on Electrical Engineering and Computer Science, Bern, Switzerland, 2018, pp. 558-563.
- [8] R. Gao, T. Pan, J. Li, D. Yao, and J. Yang, "Violence Detection Using a Multi-scale Convolutional Neural Network," in Proc. Int. Conf. on Internet of Things and Machine Learning (IoTML), Liverpool, UK, 2021, Article 3, pp. 1-7.